清华 AI 校园助手

thu-ai-assistant

AI 原生能力说明报告

模块三：AI 原生能力说明

清华大学

2026 年春季学期

2026 年 5 月 1 日

目录

1 AI 核心能力 **3**
1.1 大模型对话（LLM Chat） 3
1.2 Agent / Function Calling 3
1.3 结构化输出 4
1.4 意图识别与路由 5
1.5 确认协议（Confirmation Protocol） 5
1.6 未使用的 AI 能力 6

2 AI 如何解决痛点 **6**
2.1 目标用户与核心痛点 6
2.2 痛点与 AI 解决方案的对照 6
2.3 去掉 AI，这个产品还能成立吗？ 7
2.3.1 诚实说明：不依赖 AI 也能实现的部分 7

3 AI 技术方案 **8**
3.1 模型与 API 8
3.2 系统架构 8
3.3 AI 工作流程 9
3.4 关键技术细节 10
3.4.1 工具注册机制 10
3.4.2 ReAct 循环 11
3.4.3 认证链路 12
3.4.4 体育场馆 API 签名算法 12
3.4.5 Pending Action 确认协议 13
3.5 未来工作 14

1 AI 核心能力

thu-ai-assistant（清华 AI 校园助手）是一个以大语言模型为核心驱动的校园信息聚合平台。本章列举产品所使用的全部 AI 能力，并为每项能力给出具体的使用场景。

1.1 大模型对话（LLM Chat）

产品基于智谱 **GLM-4-Flash** 模型，通过 BigModel 开放平台 API 实现多轮对话。用户以自然语言输入查询需求，模型理解语义后生成回复或决策调用工具。

模型

智谱 GLM-4-Flash（`glm-4-flash`）

调用方式

HTTP POST，兼容 OpenAI Chat Completions 格式

多轮上下文

后端维护每个会话的消息历史（`system` + `user` + `assistant` + `tool`），实现跨轮次的上下文理解

场景示例：用户输入" 帮我查一下校园卡余额"，模型理解" 校园卡余额" 意图后，自主决定调用 `get_card_info` 工具获取数据，再将结果格式化为信息卡片返回给用户。

1.2 Agent / Function Calling

产品采用 **Agent 架构**，LLM 作为决策中枢，通过 Function Calling 机制自主选择并调用工具函数。系统注册了 **50 个工具函数**（15 个核心工具 + 35 个扩展工具），覆盖课程表、体育场馆、成绩、校园卡、电费、图书馆、新闻、教学日历、教室、选课、校园网等全部校园信息维度。

工具调用遵循 **ReAct**（Reasoning + Acting）循环模式：

1. LLM 接收用户消息，分析意图
2. LLM 返回 `tool_calls`，指定要调用的工具名称和参数
3. 后端执行工具函数，获取真实数据
4. 将工具结果以 `tool` 角色消息拼回对话历史
5. LLM 基于工具结果生成最终回复（或继续调用其他工具）

6. 最多循环 5 轮，防止无限递归

场景示例:用户说”查明天的羽毛球场馆”,LLM 自主决定调用 `get_sports_resources` 工具，传入参数 `{"date": " 明天", "sport_name": " 羽毛球"}`。后端解析日期、查询气膜馆/综体/西体三个场馆的预约数据，返回结构化结果后，LLM 将其格式化为 Markdown 表格呈现给用户。

1.3 结构化输出

通过精心设计的 **System Prompt**，约束 LLM 按照预定义的格式规范输出结果。不同查询类型对应不同的输出模板：

- **体育场馆：**Markdown 表格（场馆 | 场地 | 时段 | 状态 | 费用），用 ✓/× 标记可预约状态
- **课程表：**按星期的网格表格（节次 × 星期），空闲用”-” 表示
- **成绩：**按学期分组的表格（课程 | 学分 | 成绩 | 绩点），末尾附 GPA 汇总
- **余额类：**信息卡片格式（余额、状态等关键字段）
- **新闻：**编号列表（标题 + 日期）

System Prompt 还规定了通用规则：金额统一用 ¥ 前缀保留两位小数、时间用 HH:MM 格式、不输出原始 JSON、回复开头用一句话概括结果等。以下是 System Prompt 中输出格式规范的片段：

```
## 输出格式规范（严格遵守）
### 体育场馆查询
先给出一句话总结，然后用 Markdown 表格展示：
| 场馆 | 场地 | 时段 | 状态 | 费用 |
- 可预约用 checkmark，已满用 X
- 如果所有场馆均无可预约时段，直接说明原因
### 通用规则
- 回复开头用一句话概括结果
- 金额统一用 yen 前缀，保留两位小数
- 不要输出工具返回的原始 JSON
- 直接给结论，不要用疑问句标题
```

Listing 1: System Prompt 输出格式规范片段（`src/agent/prompt.ts`）

场景示例：用户查询成绩时，LLM 不会返回工具的原始 JSON 数据，而是自动按照 System Prompt 中定义的成绩表格模板，输出带学期分组和 GPA 汇总的规范表格。

1.4 意图识别与路由

LLM 充当**智能路由器**，自动识别用户查询的类型并路由到对应的工具函数。这一能力完全由模型的语义理解驱动，无需预定义的意图分类器或关键词匹配规则。

表 1: 意图识别与工具路由示例

用户输入	识别意图	路由工具
”查一下课表”	课程表查询	`get_schedule`
”明天羽毛球有空位吗”	体育场馆预约查询	`get_sports_resources`
”我的绩点多少”	成绩查询	`get_report`
”校园卡还有多少钱”	校园卡余额	`get_card_info`
”充值 50 块”	校园卡充值	`recharge_campus_card`
”宿舍电费还剩多少”	电费查询	`get_electricity`
”北馆三楼有空座吗”	图书馆座位查询	`get_library_seats`
”帮我预约研读间”	研读间预约	`prepare_library_room_booking`

1.5 确认协议（Confirmation Protocol）

对于充值、预约、取消等**不可逆操作**，产品实现了统一的 **Pending Action 确认协议**，确保 Agent 不会在用户未明确同意的情况下执行真实操作。协议流程如下：

1. **Prepare 阶段:**用户表达操作意图（如”充值 50 元”）,Agent 调用 `recharge_campus_card` 等 prepare 工具，创建一个待确认动作（Pending Action），返回摘要、风险等级和确认 token

2. **Awaiting Confirmation 阶段：**Agent 向用户复述操作摘要和风险，询问是否确认

3. **Confirm 阶段:**用户明确说”确认”/”执行”后,Agent 调用 `confirm_pending_action`，传入 token 执行真实操作

4. **Execute 阶段：**后端执行真实操作（创建支付订单、提交预约等），返回结果

该协议覆盖的操作类型包括：校园卡充值（微信/支付宝）、电费充值（支付宝）、体育场馆预约页面打开、图书馆研读间预约等。每个 Pending Action 有过期时间，超时未确认则自动失效。

场景示例：用户说”帮我充值 50 元”，Agent 调用 `recharge_campus_card` 创建待确认动作，回复”即将为您充值 50 元到校园卡（微信支付），确认执行吗？”。用户回复”确认”后，Agent 调用 `confirm_pending_action` 创建真实支付订单，前端展示支付二维码。

1.6 未使用的 AI 能力

为完整起见，以下列出当前产品**未使用**的 AI 能力及原因：

- **Embedding / 向量检索**：当前所有数据均通过 API 实时获取，无需本地知识库或语义检索。未来若接入校内文档问答（如培养方案解读），可引入 RAG 架构。
- **RAG（检索增强生成）**：同上，当前产品定位为" 校园信息操作助手" 而非" 知识问答系统"，数据来源为结构化 API 而非非结构化文档。
- **多模态（图像/语音）**：当前仅支持文本交互。校历图片查询虽然返回图片 URL，但图片理解能力未使用。
- **微调（Fine-tuning）**：使用通用 GLM-4-Flash 模型，通过 System Prompt 和 Function Calling 实现领域适配，未进行模型微调。

2 AI 如何解决痛点

2.1 目标用户与核心痛点

thu-ai-assistant 的目标用户是**清华大学在校学生**（本科生和研究生）。在日常校园生活中，学生面临以下核心痛点：

1. **校园信息系统高度分散**：清华校内存在多个独立的信息系统——info 信息门户、sports 体育场馆预约、学堂在线、图书馆系统、校园卡系统、电费系统、选课系统等，每个系统有独立的 URL、独立的界面和独立的操作逻辑，学生需要分别登录和操作。
2. **体育场馆预约流程繁琐**：预约羽毛球场需要手动浏览气膜馆、综体、西体等多个场馆页面，逐一检查每个时段的空闲状态，操作步骤多且耗时。
3. **日常查询效率低**：查询校园卡余额、电费余额、课程表、成绩等日常信息，需要记住各系统的 URL 和操作路径，打开多个不同的网页。
4. **操作缺乏智能引导**：传统 GUI 系统只提供固定的表单和按钮，无法根据用户的自然语言描述自动理解需求并执行操作。

2.2 痛点与 AI 解决方案的对照

表 2 展示了每个痛点如何被 AI 能力解决。

表 2: 用户痛点与 AI 解决方案对照

痛点	传统方式	AI 解决方案	涉及 AI 能力
系统分散，需分别登录	记住 5+ 个系统 URL，逐一打开并登录	统一自然语言入口，一次登录后 Agent 自动路由到对应系统的 API	意图识别与路由、Agent 工具调用
场馆预约繁琐	手动浏览 3+ 个场馆页面，逐一检查时段	一句" 查明天羽毛球"，Agent 自动查询所有场馆并汇总为表格	Agent 工具调用、结构化输出
日常查询效率低	打开对应网页 → 找到入口 → 点击查询	" 校园卡余额" 一句话直达结果	LLM 对话、意图识别
操作缺乏智能引导	手动填写表单、选择选项	Agent 理解自然语言意图，自动准备参数，通过确认协议安全执行	确认协议、Agent 工具调用

2.3 去掉 AI，这个产品还能成立吗？

答案是：不能。去掉 AI，产品将严重退化为一个传统的校园信息聚合网站，失去其核心价值。论证如下：

1. **Agent 工具调用是核心交互范式，不是锦上添花。**产品的 50 个工具函数全部通过 LLM 的 Function Calling 机制触发。没有 LLM，这些工具无法被自然语言驱动，用户需要回到传统 GUI 逐个系统操作——这正是产品要解决的痛点本身。

2. **意图识别依赖 LLM 的语义理解。**用户输入" 明天羽毛球有空位吗" 和" 查一下课表"，系统需要理解这是两种完全不同的查询类型并路由到不同工具。传统方式需要构建复杂的 NLU 管线（意图分类器 + 实体提取器 + 对话管理器），而 LLM 通过 Function Calling 一步到位。

3. **确认协议依赖 LLM 的语义理解。**Agent 需要判断用户说" 确认"" 算了"" 换一个时段" 分别对应什么操作。这种灵活的自然语言理解能力无法用简单的关键词匹配替代。

4. **结构化输出依赖 LLM 的生成能力。**将工具返回的原始数据转化为用户友好的 Markdown 表格、信息卡片等格式，需要 LLM 的文本生成能力。传统方式需要为每种查询类型编写固定的模板渲染逻辑。

2.3.1 诚实说明：不依赖 AI 也能实现的部分

需要诚实指出，部分**底层数据获取**功能本身不需要 AI：

- 校园卡余额查询、电费查询等单一 API 调用，传统网页也能实现
- 课程表数据的获取和展示，传统 App 也能完成

但 AI 为这些功能提供了**统一的自然语言入口**和**跨系统的无缝体验**。用户不需要知道" 校园卡余额在哪个系统查"，只需要用自然语言描述需求。这种" 一个入口，万事可查" 的体验，是 AI 带来的本质性提升。

3 AI 技术方案

3.1 模型与 API

thu-ai-assistant 使用智谱 AI 的 **GLM-4-Flash** 模型，通过 BigModel 开放平台 API 进行调用。表 3 列出了模型和 API 的关键信息。

表 3: 模型与 API 信息

项目	详情
模型名称	`glm-4-flash`
提供商	智谱 AI（BigModel 开放平台）
API 端点	https://open.bigmodel.cn/api/paas/v4/chat/completions
认证方式	Bearer Token（API Key）
调用方式	HTTP POST，JSON 格式
核心特性	支持 Function Calling / `tool_calls`
温度参数	0.7
最大输出	2048 tokens

调用时，系统将完整的消息列表（包含 System Prompt、用户历史消息、工具调用结果）和工具定义列表一并发送给 GLM API，设置 `tool_choice: "auto"` 让模型自主决定是否调用工具。

3.2 系统架构

图 1 展示了 thu-ai-assistant 的整体系统架构。

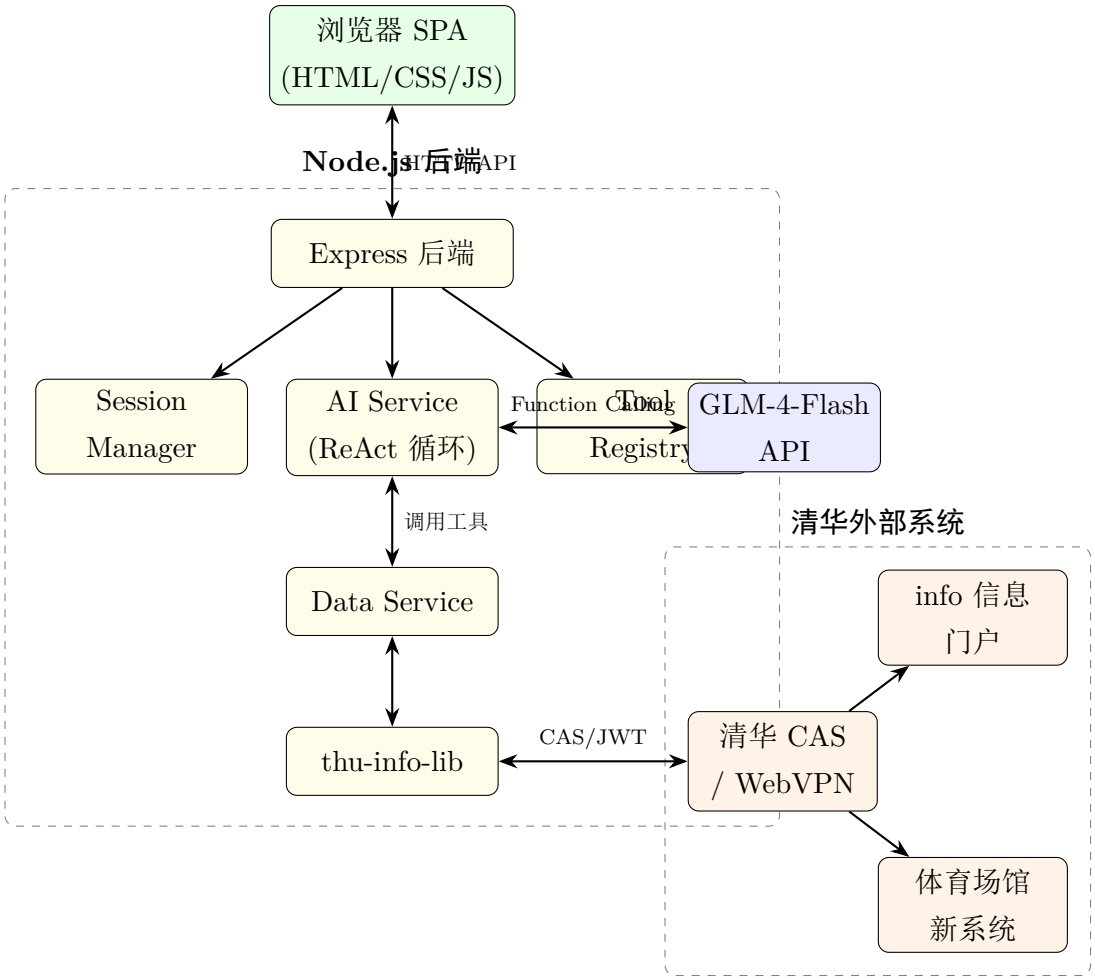

图 1: thu-ai-assistant 系统架构

系统分为三层：

- **前端层**：纯静态 SPA 页面，通过 HTTP API 与后端通信，负责渲染对话界面、支付二维码、滑块验证码面板等。
- **后端层**：基于 Express 的 Node.js 服务，核心组件包括 Session Manager（会话管理和 Pending Action 存储）、AI Service（ReAct 循环引擎）、Tool Registry（50 个工具函数注册表）、Data Service（数据获取和格式化）、thu-info-lib（清华信息系统底层库）。
- **外部系统**：GLM-4-Flash API（AI 推理）、清华统一认证系统（CAS/WebVPN）、体育场馆新系统（JWT + API 签名）、info 信息门户等。

3.3 AI 工作流程

图 2 展示了一次完整的用户交互中 AI 的工作流程。

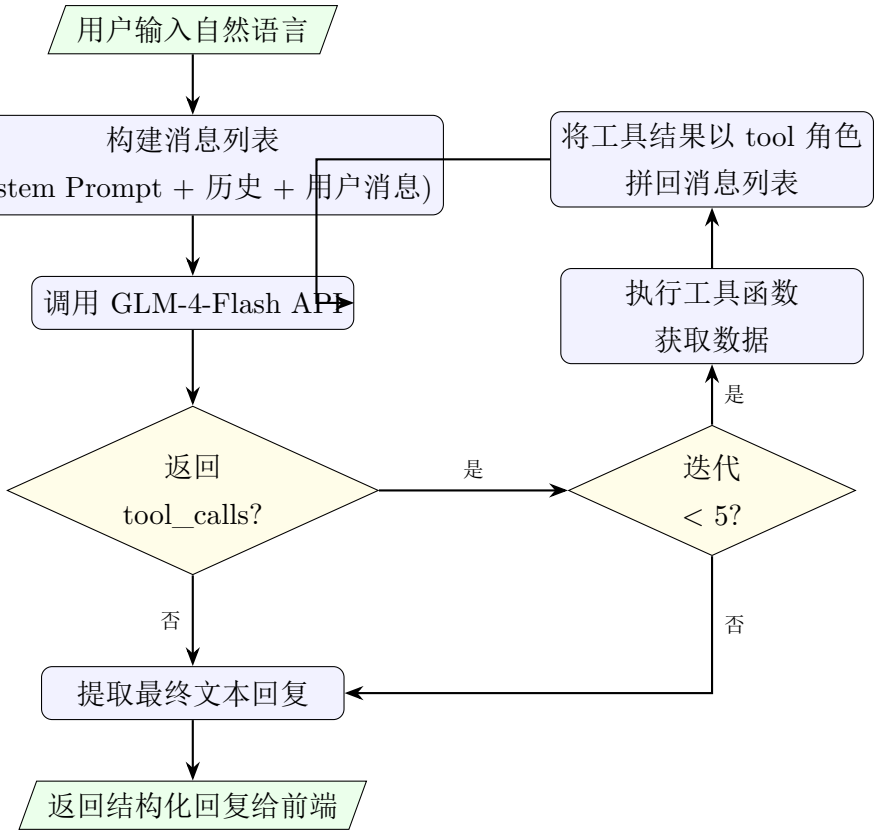

图 2: ReAct 工具调用循环流程

完整流程如下：

1. 用户在前端输入自然语言，前端 POST 到 `/api/chat`
2. AI Service 构建消息列表：System Prompt（约 2500 字符的格式规范和能力描述）+ 历史对话 + 当前用户消息
3. 调用 GLM-4-Flash API，附带 50 个工具定义
4. 解析响应：若 `finish_reason` 为 `tool_calls`，进入工具执行循环
5. 串行执行每个工具调用，将结果以 `tool` 角色消息拼回
6. 再次调用 GLM API，重复步骤 4–5，最多 5 轮
7. 获得最终文本回复，返回前端渲染

3.4 关键技术细节

3.4.1 工具注册机制

所有工具通过统一的 `AgentTool` 接口注册，每个工具包含两部分：

```
export interface AgentTool {
    definition: {
        type: "function";
        function: {
            name: string;
            description: string;
            parameters: Record<string, unknown>;
        };
    };
    run: (ctx: ToolContext, args: any) => Promise<unknown>;
}
```

Listing 2: AgentTool 接口定义（src/agent/tools/types.ts）

definition 遵循 OpenAI Function Calling 的 JSON Schema 规范，包含工具名称、描述和参数定义;run 是实际执行函数。工具注册表在 src/agent/tools/index.ts 中维护，包含 15 个核心工具和 35 个扩展工具。

3.4.2 ReAct 循环

AI Service 的核心是一个最多 5 轮的 ReAct 循环：

```
// 处理工具调用循环（最多 5 轮）
let iterations = 0;
while (assistantMessage.tool_calls && iterations < 5) {
    iterations++;
    fullMessages.push({
        role: "assistant",
        content: assistantMessage.content || "",
        tool_calls: assistantMessage.tool_calls,
    });
    for (const toolCall of assistantMessage.tool_calls) {
        const args = JSON.parse(toolCall.function.arguments);
        const result = await executeTool(
            helper, toolCall.function.name, args, sessionId
        );
        fullMessages.push({
            role: "tool", content: result,
            tool_call_id: toolCall.id,
        });
```

```
    }
    response = await callGLM(fullMessages);
    assistantMessage = response.choices[0].message;
}
```

Listing 3: ReAct 循环核心逻辑（`src/agent/ai-service.ts`）

每轮中，模型可以返回多个 `tool_calls`，系统串行执行后将所有结果拼回消息历史，再次调用 GLM 获取下一步决策。

3.4.3 认证链路

thu-ai-assistant 需要访问清华多个内部系统，认证链路如下：

1. **WebVPN 登录**：通过 `webvpn.tsinghua.edu.cn` 建立 VPN 隧道，使用用户的学号和密码进行 CAS 认证
2. **CAS 漫游**：通过 `roam` 函数（`packages/thu-info-lib/src/lib/core.ts`）将 WebVPN 会话漫游到目标系统（info、sports 等）
3. **体育系统 JWT Token**：对于体育场馆新系统，需要额外的 CAS 登录流程获取 JWT token：
   - 获取 CAS 地址 → CAS 重定向 → id.tsinghua.edu.cn 登录 → 回调获取 uniToken → 换取 JWT token
4. **API 签名**：体育场馆新系统的每个 API 请求需要携带签名参数

3.4.4 体育场馆 API 签名算法

体育场馆新系统采用 MD5 签名机制保护 API 接口。签名公式为：

$$\text{sign} = \text{MD5}(\texttt{"appId="}+\text{appId}+\texttt{"\&nonce="}+\text{nonce}+\texttt{"\&timeStamp="}+\text{ts}+\texttt{"\&key="}+\text{KEY}) \tag{1}$$

其中：

- `appId` = `"1497016617475903488"`（固定值）
- `nonce` = 32 位随机字符串
- `ts` = 当前时间戳（毫秒）
- `KEY` = 通过逆向前端 JavaScript 代码获取的三段拼接密钥

密钥 KEY 的获取过程：前端 JS 中的 `getKeys()` 函数通过字符索引、数组翻转、奇偶位拆分等混淆手段生成三段子密钥，拼接后得到完整密钥。我们通过逆向还原了该算法（见 `src/verify-sign.ts`），并验证了签名计算的正确性。

```
function getSign(timeStamp?: number, nonce?: string) {
    const appId = "1497016617475903488";
    const key = getKeys().join("");
    const ts = timeStamp ?? Date.now();
    const nc = nonce ?? randomString(32);
    const raw = `appId=${appId}&nonce=${nc}`
              + `&timeStamp=${ts}&key=${key}`;
    const sign = md5(raw);
    return { appId, timeStamp: ts, nonce: nc, sign, key };
}
```

Listing 4: 签名计算函数（`src/verify-sign.ts`）

3.4.5 Pending Action 确认协议

对于不可逆操作，系统实现了四阶段确认协议：

表 4: Pending Action 确认协议四阶段

阶段	名称	说明
1	Prepare	用户表达操作意图，Agent 调用 `prepare_*` 工具创建待确认动作，生成唯一 `confirmation_token`
2	Awaiting	Agent 向用户复述操作摘要（summary）、风险等级（risk）和过期时间（expires_at），询问确认
3	Confirm	用户明确确认，Agent 调用 `confirm_pending_action` 并传入 token
4	Execute	系统执行真实操作（创建支付订单/提交预约/打开页面），返回执行结果

该协议已覆盖以下操作类型：校园卡充值（`campus_card_recharge`）、电费充值（`electricity_recharge`）、体育场馆预约页打开（`open_sports_booking_page`）、研读间预约（`library_room_booking`）。每个待确认动作有 5 分钟有效期，过期自动失效，防止误操作。

3.5 未来工作

当前版本仍有以下改进空间：

1. **滑块验证码自动化：**体育场馆预约的最后一步需要用户手动完成滑块验证码，未来可探索自动化方案。
2. **更多场馆类型支持：**当前主要支持羽毛球、篮球等常见场馆，可扩展到游泳馆、健身房等。
3. **RAG 知识库增强：**引入校内文档（培养方案、选课手册、校规校纪等）的向量检索，增强 Agent 的知识覆盖。
4. **多模型切换：**支持在 GLM-4-Flash、GPT-4o-mini 等多个模型间切换，提供更灵活的推理能力选择。
5. **主动通知：**基于用户偏好，在场馆有空位、成绩发布等事件发生时主动推送通知。
6. **更多不可逆操作接入：**将选课、退课、取消预约等操作纳入确认协议，实现完整的校园事务自动化。

参考文献

[1] 智谱 AI. *GLM-4 系列模型 API 文档*. https://open.bigmodel.cn/dev/api/normal-model/glm-4, 2024.

[2] Yao, S., Zhao, J., Yu, D., et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, 2022.

[3] OpenAI. *Function Calling and Other API Updates*. https://openai.com/blog/function-calling-and-other-api-updates, 2023.

[4] thu-info-lib. *清华大学信息系统非官方 TypeScript 库*. https://github.com/thu-info-community/thu-info-app, 2024.